Lab Assignment-12.4

Hall.No:2303A51873

Name:R.Vineesha

Batch:14

Task 1: Bubble Sort for Ranking Exam Scores

Scenario

You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.

Task Description

• Implement Bubble Sort in Python to sort a list of student scores.

• Use an AI tool to:

o Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes

o Identify early-termination conditions when the list becomes sorted

o Provide a brief time complexity analysis

Expected Outcome

• A Bubble Sort implementation with:

o AI-generated comments explaining the logic

o Clear explanation of best, average, and worst-case complexity

o Sample input/output showing sorted scores

Prompt

Generate a Python program to implement Bubble Sort to sort a list of student exam scores in ascending order. Include inline comments explaining comparisons, swaps, and iteration passes. Add an early termination condition to stop when the list is already sorted. Display original and sorted scores with sample input/output and provide time complexity analysis for best, average, and worst cases.

Code

```python
#Generate a Python program to implement Bubble Sort for sorting student exam scores in ascending order. Include clear inline comments explaining comparisons, swaps, and
# Lab 12 - Algorithms with AI Assistance
# Task 1: Bubble Sort for Ranking Exam Scores

def bubble_sort(scores):
    n = len(scores)  # Total number of scores

    # Traverse through all elements
    for i in range(n):

        swapped = False  # To track whether any swap happened

        # Last i elements are already sorted
        for j in range(0, n - i - 1):

            # Compare adjacent elements
            if scores[j] > scores[j + 1]:

                # Swap if elements are in wrong order
                scores[j], scores[j + 1] = scores[j + 1], scores[j]

                swapped = True  # Mark that swap occurred

        # Early termination: stop if no swaps happened
        if not swapped:
            print("Early termination: List is already sorted.")
            break

    return scores


# Sample Input: Student exam scores
scores = [78, 45, 90, 62, 84, 55]

print("Original Scores:")
print(scores)

# Call Bubble Sort function
sorted_scores = bubble_sort(scores)

print("\nSorted Scores in Ascending Order:")
print(sorted_scores)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/Untitled-1.py"
Original Scores:
[78, 45, 90, 62, 84, 55]
Early termination: List is already sorted.

Sorted Scores in Ascending Order:
[45, 55, 62, 78, 84, 90]
PS D:\AI coding\lab12.4>
```

Time Complexity Analysis

Case	Time Complexity	Explantion
Best Case	O(n)	When list is already sorted (early termination)
Average Case	$O(n^2)$	When elements are randomly ordere
Worst Case	$O(n^2)$	When list is in reverse order

Observation

The Bubble Sort algorithm was implemented in Python to sort a list of student exam scores in ascending order. The algorithm repeatedly compared adjacent elements and swapped them if they were in the wrong order. After each pass, the largest unsorted element moved to its correct position. An optimization using a swapped flag was included to detect if no swaps occurred during a pass, allowing early termination when the list was already sorted. The program successfully displayed the original scores and the sorted scores. It was observed that Bubble Sort works correctly for small datasets and improves efficiency when early termination is applied.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario

You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates.

Task Description

• Start with a Bubble Sort implementation.
• Ask AI to:
o Review the problem and suggest a more suitable sorting algorithm
o Generate an Insertion Sort implementation
o Explain why Insertion Sort performs better on nearly sorted data
• Compare execution behavior on nearly sorted input

Expected Outcome

• Two sorting implementations:
o Bubble Sort
o Insertion Sort
• AI-assisted explanation highlighting efficiency differences for partially sorted datasets

Prompt

Generate Python programs for Bubble Sort and Insertion Sort to sort nearly sorted student attendance roll numbers. Suggest the most suitable algorithm for nearly sorted data, explain why it performs better, and compare the execution behavior of both algorithms with sample input and output.

Code

Untitled-1.py

Untitled-1.py > ...

```python
#Generate Python programs for Bubble Sort and Insertion Sort to sort nearly sorted student attendance roll numbers. Sugges
# Lab 12 – Task 2: Improving Sorting for Nearly Sorted Attendance Records

# Bubble Sort Implementation
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False  # Track swaps

        for j in range(0, n - i - 1):

            # Compare adjacent roll numbers
            if arr[j] > arr[j + 1]:

                # Swap if wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Early termination if already sorted
        if not swapped:
            break

    return arr


# Insertion Sort Implementation
def insertion_sort(arr):

    # Traverse from second element
    for i in range(1, len(arr)):

        key = arr[i]  # Element to insert
        j = i - 1

        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Place key in correct position
        arr[j + 1] = key

    return arr


# Nearly sorted attendance roll numbers
attendance = [101, 102, 103, 105, 104, 106, 107]

print("Original Attendance Roll Numbers:")
print(attendance)

# Bubble Sort
bubble_result = bubble_sort(attendance.copy())
print("\nBubble Sort Result:")
print(bubble_result)

# Insertion Sort
insertion_result = insertion_sort(attendance.copy())
print("\nInsertion Sort Result:")
print(insertion_result)
```

Output

```
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/Untitled-1.py"
Original Attendance Roll Numbers:
[101, 102, 103, 105, 104, 106, 107]

Bubble Sort Result:
[101, 102, 103, 104, 105, 106, 107]

Insertion Sort Result:
[101, 102, 103, 104, 105, 106, 107]
PS D:\AI coding\lab12.4>
```

AI-Assisted Explanation: Best Algorithm Suggestion

AI suggested Insertion Sort as more suitable because the data is nearly sorted.

Reason:
- Insertion Sort only shifts misplaced elements
- It does not repeatedly compare already sorted elements
- It performs fewer operations

Why Insertion Sort Performs Better on Nearly Sorted Data

Insertion Sort is efficient because:

- It only moves incorrect elements to correct position
- Already sorted elements are skipped quickly
- Requires fewer comparisons and shifts

Example:
Only 105 and 104 need adjustment, so minimal work is required.

Observation

It was observed that both Bubble Sort and Insertion Sort correctly sorted the nearly sorted attendance roll numbers. However, Insertion Sort required fewer operations because it directly placed misplaced elements in their correct position. Bubble Sort performed more comparisons even though the list was almost sorted. Therefore, Insertion Sort showed better efficiency for nearly sorted data.

Task 3: Searching Student Records in a Database

Scenario

You are developing a student information portal where users search for student records by roll number.

Task Description

• Implement:

o Linear Search for unsorted student data

o Binary Search for sorted student data

• Use AI to:

o Add docstrings explaining parameters and return values

o Explain when Binary Search is applicable

o Highlight performance differences between the two searches

Expected Outcome

• Two working search implementations with docstrings

• AI-generated explanation of:

o Time complexity

o Use cases for Linear vs Binary Search

• A short student observation comparing results on sorted vs unsorted lists

Prompt

Generate Python programs to implement Linear Search and Binary Search for searching student roll numbers. Add proper docstrings explaining parameters and return values. Explain when Binary Search can be used and compare performance and time complexity of Linear and Binary Search with examples.

Code

```python
#explaining parameters and return values. Explain when Binary Search can be used and compare performance and time complexity c
# Lab 12 – Task 3: Searching Student Records in a Database

# Linear Search Implementation
def linear_search(arr, key):
    """
    Performs Linear Search on an unsorted list.

    Parameters:
    arr (list): List of student roll numbers
    key (int): Roll number to search

    Returns:
    int: Index of the roll number if found
    -1: If roll number is not found
    """

    # Traverse entire list
    for i in range(len(arr)):

        # Check if current element matches key
        if arr[i] == key:
            return i

    return -1


# Binary Search Implementation
def binary_search(arr, key):
    """
    Performs Binary Search on a sorted list.

    Parameters:
    arr (list): Sorted list of student roll numbers
    key (int): Roll number to search

    Returns:
    int: Index of the roll number if found
    -1: If roll number is not found
    """

```

```python
def binary_search(arr, key):

    while low <= high:

        mid = (low + high) // 2

        # Check if middle element is key
        if arr[mid] == key:
            return mid

        # If key is greater, search right half
        elif arr[mid] < key:
            low = mid + 1

        # If key is smaller, search left half
        else:
            high = mid - 1

    return -1


# Unsorted student roll numbers
unsorted_roll = [105, 102, 108, 101, 104, 107]

# Sorted student roll numbers
sorted_roll = sorted(unsorted_roll)

key = 104

print("Unsorted Roll Numbers:", unsorted_roll)
print("Sorted Roll Numbers:", sorted_roll)


# Linear Search on unsorted list
result_linear = linear_search(unsorted_roll, key)

if result_linear != -1:
    print("\nLinear Search: Roll number found at index", result_linear)
else:
    print("\nLinear Search: Roll number not found")


# Binary Search on sorted list
result_binary = binary_search(sorted_roll, key)

if result_binary != -1:
    print("Binary Search: Roll number found at index", result_binary)
else:
    print("Binary Search: Roll number not found")
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

[101, 102, 103, 104, 105, 106, 107]
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/Untitled-1.py"
Unsorted Roll Numbers: [105, 102, 108, 101, 104, 107]
Sorted Roll Numbers: [101, 102, 104, 105, 107, 108]

Linear Search: Roll number found at index 4
Binary Search: Roll number found at index 2
PS D:\AI coding\lab12.4>
```

AI Explanation: When Binary Search is Applicable

Binary Search can only be used when:

- The list is sorted
- Random access to elements is possible

Binary Search works by dividing the list into halves repeatedly.

Not suitable for unsorted data.

Time Complexity Analysis

Linear Search:
- Best Case: O(1)
- Average Case: O(n)
- Worst Case: O(n)

Binary Search:
- Best Case: O(1)
- Average Case: O(log n)
- Worst Case: O(log n)

Binary Search is much faster for large datasets.

Observation

It was observed that Linear Search successfully found the roll number in the unsorted list by checking each element sequentially. Binary Search found the roll number faster in the sorted list by repeatedly dividing the search space into halves. Binary Search required fewer comparisons and showed better performance compared to Linear Search. Therefore, Binary Search is more efficient for searching in sorted datasets, while Linear Search is suitable for unsorted data.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario

You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Task Description

• Provide AI with partially written recursive functions for:
o Quick Sort
o Merge Sort
• Ask AI to:
o Complete the recursive logic
o Add meaningful docstrings
o Explain how recursion works in each algorithm
• Test both algorithms on:
o Random data
o Sorted data
o Reverse-sorted data

Expected Outcome

• Fully functional Quick Sort and Merge Sort implementations
• AI-generated comparison covering:
o Best, average, and worst-case complexities

o Practical scenarios where one algorithm is preferred over the other

Prompt

Generate Python implementations of Quick Sort and Merge Sort using recursion. Complete the recursive logic, add meaningful docstrings explaining parameters and return values, and explain how recursion works in both algorithms. Test both algorithms on random, sorted, and reverse-sorted datasets, and compare their time complexity and practical use cases.

Code

```python
#Generate Python implementations of Quick Sort and Merge Sort using recursion. Complete the recursive logic, add meaningful docstrings explaining parameters and return
# Lab 12 - Task 4: Quick Sort vs Merge Sort for Data Processing

# Quick Sort Implementation
def quick_sort(arr):
    """
    Performs Quick Sort using recursion.

    Parameters:
    arr (list): List of elements to sort

    Returns:
    list: New sorted list
    """

    # Base case: if list has 0 or 1 element, it is already sorted
    if len(arr) <= 1:
        return arr

    # Select pivot element (last element)
    pivot = arr[-1]

    # Elements smaller than pivot
    left = []

    # Elements greater than pivot
    right = []

    # Divide elements based on pivot
    for i in range(len(arr) - 1):
        if arr[i] <= pivot:
            left.append(arr[i])
        else:
            right.append(arr[i])

    # Recursive calls
    return quick_sort(left) + [pivot] + quick_sort(right)


# Merge Sort Implementation
def merge_sort(arr):
    """
    Performs Merge Sort using recursion.

    Parameters:
    arr (list): List of elements to sort

    Returns:
    list: Sorted list
    """
```

```python
def merge_sort(arr):
    """
    list: Sorted list
    """

    # Base case
    if len(arr) <= 1:
        return arr

    # Divide array into two halves
    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Merge sorted halves
    return merge(left, right)


def merge(left, right):
    """
    Merges two sorted lists into one sorted list.
    """

    result = []
    i = j = 0

    # Compare elements and merge
    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result


# Test datasets
random_data = [45, 12, 78, 34, 23, 90, 11]
sorted_data = [11, 12, 23, 34, 45, 78, 90]
reverse_data = [90, 78, 45, 34, 23, 12, 11]

print("Random Data:", random_data)
print("Quick Sort:", quick_sort(random_data))
print("Merge Sort:", merge_sort(random_data))

print("\nSorted Data:", sorted_data)
print("Quick Sort:", quick_sort(sorted_data))
print("Merge Sort:", merge_sort(sorted_data))

print("\nReverse Sorted Data:", reverse_data)
print("Quick Sort:", quick_sort(reverse_data))
print("Merge Sort:", merge_sort(reverse_data))
```

Output

```
Linear Search: Roll number found at index 4
Binary Search: Roll number found at index 2
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/Untitled-1.py"
Random Data: [45, 12, 78, 34, 23, 90, 11]
Quick Sort: [11, 12, 23, 34, 45, 78, 90]
Merge Sort: [11, 12, 23, 34, 45, 78, 90]

Sorted Data: [11, 12, 23, 34, 45, 78, 90]
Quick Sort: [11, 12, 23, 34, 45, 78, 90]
Merge Sort: [11, 12, 23, 34, 45, 78, 90]

Reverse Sorted Data: [90, 78, 45, 34, 23, 12, 11]
Quick Sort: [11, 12, 23, 34, 45, 78, 90]
Merge Sort: [11, 12, 23, 34, 45, 78, 90]
PS D:\AI coding\lab12.4>
```

AI Explanation: How Recursion Works

Quick Sort Recursion

- Select pivot element
- Divide array into left and right parts
- Recursively sort left and right
- Combine results

Merge Sort Recursion

- Divide array into halves
- Recursively sort each half
- Merge sorted halves

Time Complexity Comparison

Case	Quick sort	Explnation
Best Case	O(n log n)	O(n log n)
Average Case	O(n log n)	O(n log n)
Worst Case	$O(n^2)$	O(n log n)

Observation

It was observed that both Quick Sort and Merge Sort successfully sorted random, sorted, and reverse-sorted datasets. Quick Sort performed faster in most cases due to efficient partitioning but showed slower performance in worst-case scenarios such as already sorted data. Merge Sort maintained consistent performance for all datasets because it always divides the list equally. Therefore, Quick Sort is suitable for general purposes, while Merge Sort is preferred for large datasets requiring stable and reliable performance.

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.

Task Description

- Write a naive duplicate detection algorithm using nested loops.
- Use AI to:
  - Analyze the time complexity
  - Suggest an optimized approach using sets or dictionaries
  - Rewrite the algorithm with improved efficiency
- Compare execution behavior conceptually for large input sizes

Expected Outcome

- Two versions of the algorithm:
  - Brute-force ($O(n^2)$)
  - Optimized ($O(n)$)
- AI-assisted explanation showing how and why performance improved

Prompt

Generate Python programs to detect duplicate user IDs using a brute-force nested loop approach and an optimized approach using sets. Analyze the time complexity of both methods and explain how the optimized version improves performance for large datasets.

Code

```python
#Generate Python programs to detect duplicate user IDs using a brute-force nested loop approach and an optimized approach using sets. Analyze the time complexity of both methods and explain how the optimized version improves performance for large datasets.
# Lab 12 - Task 5: Optimizing Duplicate Detection Algorithm

# Brute-force duplicate detection using nested loops
def find_duplicates_bruteforce(user_ids):
    """
    Detect duplicates using nested loops (O(n²)).

    Parameters:
    user_ids (list): List of user IDs

    Returns:
    list: List of duplicate IDs
    """

    duplicates = []
    n = len(user_ids)

    # Compare each element with every other element
    for i in range(n):
        for j in range(i + 1, n):

            # If duplicate found
            if user_ids[i] == user_ids[j] and user_ids[i] not in duplicates:
                duplicates.append(user_ids[i])

    return duplicates


# Optimized duplicate detection using set (O(n))
def find_duplicates_optimized(user_ids):
    """
    Detect duplicates using set for faster lookup (O(n)).

    Parameters:
    user_ids (list): List of user IDs

    Returns:
    list: List of duplicate IDs
    """

    seen = set()
    duplicates = set()

    # Traverse list once
    for user in user_ids:

        # Check if already seen
        if user in seen:
            duplicates.add(user)
        else:
            seen.add(user)

    return list(duplicates)


# Sample dataset
user_ids = [101, 205, 309, 101, 405, 205, 512, 309]

print("User IDs:", user_ids)

# Brute-force method
dup1 = find_duplicates_bruteforce(user_ids)
print("\nBrute-force Duplicates:", dup1)

# Optimized method
dup2 = find_duplicates_optimized(user_ids)
print("Optimized Duplicates:", dup2)
```

Output

```
PS D:\AI coding\lab12.4> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/lab12.4/Untitled-1.py"
User IDs: [101, 205, 309, 101, 405, 205, 512, 309]

Brute-force Duplicates: [101, 205, 309]
Optimized Duplicates: [205, 309, 101]
PS D:\AI coding\lab12.4>
```

Time Complexity Analysis

Brute-force Method

Nested loops compare every element.

Time Complexity:
$O(n^2)$

Example:
For 1000 elements → 1000 × 1000 = 1,000,000 comparisons

Very slow for large datasets.

Optimized Method (Set-based)

Traverses list once.

Time Complexity:
O(n)

Example:
For 1000 elements → only 1000 operations

Much faster.

Observation

It was observed that the brute-force method successfully detected duplicate user IDs but required more comparisons due to nested loops, resulting in slower performance. The optimized method using sets detected duplicates much faster by checking each element only once. This reduced the time complexity from $O(n^2)$ to $O(n)$. Therefore, the optimized approach is more efficient and suitable for large datasets.